

Module 1 – Introduction to AI & Python | Learning Notes

Name: ARATI GAVALI

Internship: ai/ml intern

Module 1: Introduction to AI & Python

1. Introduction to Artificial Intelligence (AI)

Artificial Intelligence (AI) is a branch of computer science that enables machines to perform tasks that normally require human intelligence.

Key Points:

- AI enables machines to learn, reason and make decisions.
- It can process data and identify patterns.
- AI is used in many real-world applications.
- Examples include chatbots, voice assistants, recommendation systems and image recognition.

Example:

A chatbot uses AI to understand a user's question and generate a suitable response.

2. Introduction to Machine Learning (ML)

Machine Learning (ML) is a subset of Artificial Intelligence that enables computers to learn from data and make predictions or decisions without being explicitly programmed for every task.

Key Points:

- ML learns patterns from existing data.
- It improves its performance through experience/data.
- ML is commonly used for prediction and classification.
- Examples include spam detection, recommendation systems and fraud detection.

Example:

A spam filter learns from previous emails and predicts whether a new email is spam or not.

3. Introduction to Deep Learning (DL)

Deep Learning (DL) is a subset of Machine Learning that uses artificial neural networks with multiple layers to learn complex patterns from large amounts of data.

Key Points:

- Deep Learning is inspired by the human brain's neural networks.
- It can automatically learn useful features from data.
- It works especially well with large and complex datasets.
- It is widely used in image, speech and language-related applications.

Example:

Face recognition systems use Deep Learning to identify and recognize faces from images.

4. Difference Between AI, ML and Deep Learning

AI	ML	Deep Learning
Broad concept of making machines intelligent	Subset of AI that learns from data	Subset of ML using neural networks
Can use rules, logic and learning	Mainly learns patterns from data	Learns complex patterns from large data
Includes ML and Deep Learning	Part of AI	Part of ML
Example: AI chatbot	Example: Spam detection	Example: Face recognition

5. Real-World Applications of AI

AI is used in many areas of everyday life and different industries.

- **Chatbots:** Answer questions and interact with users.
- **Recommendation Systems:** Suggest movies, songs, products, etc.
- **Healthcare:** Helps in disease detection and medical image analysis.
- **Self-Driving Technology:** Helps vehicles understand their surroundings.
- **Voice Assistants:** Understand voice commands and perform tasks.
- **Cybersecurity:** Helps detect suspicious activities and threats.

Example:

Netflix and YouTube use recommendation systems to suggest content based on user activity.

6. Introduction to Python

Python is a high-level, interpreted and general-purpose programming language known for its simple and readable syntax.

Key Points:

- Easy to learn and beginner-friendly.
- Uses simple and readable syntax.
- Supports multiple programming paradigms.
- Has a large collection of libraries and frameworks.
- Widely used in AI, Machine Learning, Data Science and Web Development.

Why Python for AI/ML?

- Easy syntax makes development faster.
- Libraries such as **NumPy**, **Pandas** and **Scikit-learn** are widely used.
- It has a large developer community and many learning resources.

Example:

```
name = "Arati"

print("Hello", name)
```

7. Python Programming Concepts

7.1 Variable

A **variable** is a name used to store a value in a program. The value of a variable can be changed during program execution.

Key Points:

- Variables store data values.
- Python does not require specifying the data type while creating a variable.
- A variable can store different types of values.

Example:

```
name = "Arati"
```

```
age = 20
```

```
branch = "AIML"
```

```
print(name)
```

```
print(age)
```

```
print(branch)
```

7.2 Data Types

Data types define the type of value stored in a variable.

Data Type	Meaning	Example
<code>int</code>	Whole numbers	<code>age = 20</code>
<code>float</code>	Decimal numbers	<code>cgpa=9.0</code>

<code>str</code>	Text	<code>name="Arati"</code>
<code>bool</code>	True or False	<code>is_student = True</code>
<code>list</code>	Collection of values	<code>fruits = ["Apple", "Mango"]</code>
<code>dict</code>	Key-value pairs	<code>{"name": "Arati"}</code>

Example:

```
age = 20
cgpa = 9.0
name = "Arati"
is_student = True
```

```
print(type(age))
print(type(cgpa))
print(type(name))
print(type(is_student))
```

7.3 Conditional Statements

Conditional statements are used to make decisions in a Python program based on a condition.

Types:

- `if` → checks the first condition.

- **elif** → checks another condition if the previous condition is false.
- **else** → executes when all conditions are false.

Example:

```
marks = 75
```

```
if marks >= 75:
```

```
    print("Very Good")
```

```
elif marks >= 40:
```

```
    print("Pass")
```

```
else:
```

```
    print("Fail")
```

7.4 Loops

Loops are used to execute a block of code repeatedly.

Types of loops in Python:

- **for** loop → used when we want to repeat code for a sequence or a specific number of times.
- **while** loop → repeats code as long as a condition is **True**.

Example — **for loop:**

```
for i in range(1, 6):
```

```
    print(i)
```

Example — **while loop:**

```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
count += 1
```

7.5 Tuples

A tuple is an ordered collection of elements that cannot be changed after creation.

```
student = ("Arati", "AIML", 3, 8.9)
```

```
print(student)
```

```
print(student[1])
```

```
print(student[3])
```

Key Point: Tuples are immutable, meaning their elements cannot be modified.

7.6 Sets

A set is an unordered collection of unique elements. It automatically removes duplicate values.

```
numbers = {10, 20, 20, 30, 40}
```

```
print(numbers)
```

Key Point: Sets are useful when we need to store only unique values.

7.7. File Handling

File handling is used to read data from and write data to files.

Writing to a file

```
file = open("student.txt", "w")
```

```
file.write("Name: Arati\nCourse: AIML")
```

```
file.close()
```

Reading from a file

```
file = open("student.txt", "r")

content = file.read()

print(content)

file.close()
```

Key Point:

"w" → Write

"r" → Read

7.8. Exception Handling

Exception handling is used to handle runtime errors without stopping the entire program.

try:

```
number = int(input("Enter a number: "))

print(10 / number)
```

except ValueError:

```
print("Please enter a valid number.")
```

except ZeroDivisionError:

```
print("Cannot divide by zero.")
```

Key Point: `try` contains risky code, while `except` handles expected errors.

7.8. Basic OOP

Object-Oriented Programming (OOP) organizes programs using classes and objects.

class Student:

```
def __init__(self, name, course):
```

```
    self.name = name
```

```
    self.course = course
```

```
def display(self):
```

```
    print("Name:", self.name)
```

```
    print("Course:", self.course)
```

```
student1 = Student("Arati", "AIML")
```

```
student1.display()
```

Key Points:

- **Class** → blueprint for creating objects
- **Object** → instance of a class
- **Attribute** → data stored in an object
- **Method** → function defined inside a class

7.9 Functions

A function is a reusable block of code designed to perform a specific task. Functions help make programs organized, reusable and easier to understand.

Key Points:

- Functions are defined using the **def** keyword.
- They can accept parameters.
- They can return a value using **return**.
- A function is executed by calling its name.

Example:

```
def square(num):
```

```
    return num * num
```

```
result = square(5)
```

```
print(result)
```

7.6 Lists

A **list** is a collection used to store multiple values in a single variable. Lists are ordered and changeable (mutable).

Key Points:

- Lists are written using square brackets `[ ]`.
- They can contain different types of values.
- List items can be accessed using their index.
- `append()` is used to add an item.
- `remove()` is used to remove an item.

Example:

```
fruits = ["Apple", "Banana", "Mango"]
```

```
print(fruits[0])
```

```
fruits.append("Orange")
```

```
print(fruits)
```

7.7 Dictionaries

Ye paste kar:

A **dictionary** is a collection of data stored in **key-value pairs**. It is useful when we want to store related information with meaningful keys.

Key Points:

- Dictionaries are written using `{ }`.
- Data is stored as `key: value` pairs.
- Values can be accessed using their keys.
- We can add, update and remove items.

Example:

```
student = {  
    "name": "Arati",  
    "age": 20,  
    "branch": "AIML"  
}  
  
print(student["name"])  
  
student["city"] = "Pune"  
print(student)
```

15 — Beginner Python Programs

Program 1 — Calculate Sum

```
a = 10  
b = 20  
  
sum = a + b  
  
print("Sum:", sum)
```

Output: sum = 30

Program 2 — Check Pass or Fail

```
marks = 65  
  
if marks >= 40:  
    print("Pass")  
else:  
    print("Fail")
```

Output:

Pass

Program 3 — Find Square Using Function

```
def square(num):  
    return num * num  
  
print("Square:", square(5))
```

Output:

Square: 25

Program 4 — Display Items from a List

```
fruits = ["Apple", "Banana", "Mango"]
```

```
for fruit in fruits:  
    print(fruit)
```

Output:

```
Apple  
Banana  
Mango
```

9. Key Learnings

During this module, I learned the fundamentals of Artificial Intelligence, Machine Learning and Deep Learning, along with their differences and real-world applications.

I also strengthened my Python programming basics by practicing variables, data types, conditional statements, loops, functions, lists and dictionaries.

Through practical programs and projects, I learned how to apply these concepts to solve simple real-world problems.